

Building LLMs from Scratch

Generated by Bookify

Table of Contents

- 1. Introduction**
- 2. Introduction & Fundamentals**
 - | Introduction to Large Language Models
- 3. Data Preprocessing & Tokenization**
 - | Context: The Foundation of LLM Data Pipelines
- 4. The Attention Mechanism**
 - | Opening Context: The Engine of the Transformer
- 5. LLM Architecture & Components**
 - | LLM Architecture & Components: A Deep Dive into the Transformer
- 6. Pre-training & Model Evaluation**
 - | Context and Objectives
- 7. Classification Fine-tuning**
 - | Context and Fundamentals of Fine-tuning
- 8. Instruction Fine-tuning & Conclusion**
 - | 1. Context: The Transition from Pre-training to Instruction Following
- 9. Conclusion**
- 10. Glossary**
- 11. References & Resources**
 - | Introduction & Fundamentals

Introduction

In the era of generative artificial intelligence, Large Language Models (LLMs) have transitioned from academic curiosities to the engines of modern software. While many developers have learned to interact with these models via APIs, true mastery lies in understanding the intricate machinery beneath the surface. This book is designed to pull back the curtain, guiding you through the rigorous process of building a functional LLM from the ground up. We begin by establishing a solid foundation in the mathematical and computational principles of deep learning before diving into the practicalities of data preprocessing and tokenization. By transforming raw text into a format a machine can digest, you will prepare the groundwork for the core of the architecture: the attention mechanism.

As we progress, you will assemble the individual components of a transformer-based model, learning how layers of neurons and specialized blocks collaborate to predict the next word in a sequence. The journey continues into the computationally intensive phase of pre-training, where you will learn how to initialize weights and evaluate a model's nascent intelligence. Once the base model is established, we shift our focus to the art of refinement. You will explore classification fine-tuning to give your model specific analytical capabilities, followed by instruction fine-tuning to transform a simple text predictor into a helpful, conversational assistant. By the conclusion of this book, you will have moved beyond being a mere consumer of AI, possessing the skills to architect and train your own specialized models.

This book is written for software engineers, data scientists, and curious technologists who possess a working knowledge of Python and a basic grasp of machine learning concepts. You do not need a PhD in mathematics to follow along; instead, we prioritize a "code-first" approach where every theoretical concept is immediately reinforced with implementation. To get the most out of this experience, you should treat each chapter as a building block, coding along in a notebook environment as we iterate on our design. By the time you reach the final pages, you will not only understand the "how" of large language models but also the "why," giving you the confidence to innovate in a field that is reshaping the world.

Introduction & Fundamentals

Introduction to Large Language Models

The landscape of artificial intelligence has undergone a seismic shift in recent years, transitioning from specialized, rule-based systems to generalized, generative architectures. At the heart of this revolution lies the Large Language Model (LLM). While many practitioners interact with these models through high-level APIs, a true mastery of the field requires a "nuts and bolts" understanding of how these systems are constructed from the ground up [\[1\]](#).

Historically, natural language processing (NLP) relied on rigid structures. In the 1960s, systems like ELIZA attempted to simulate human conversation using pattern matching and substitution rules [\[2\]](#). These early "chatbots" lacked any true understanding of context, often leading to circular dialogues. Modern LLMs, by contrast, are built on deep neural networks designed to understand, generate, and respond to human-like text [\[3\]](#).

This chapter serves as the foundational pillar for building an LLM from scratch. We will explore the definitions, the historical evolution from simple NLP to the Transformer architecture, and the specific stages of development—pre-training and fine-tuning—that transform a raw neural network into a sophisticated digital assistant.

Defining the Large Language Model

To understand an LLM, one must deconstruct the term into its constituent parts: **Large**, **Language**, and **Model**.

What Makes it a "Model"?

At its core, an LLM is a **Deep Neural Network** [\[3\]](#). A neural network is a computational structure inspired by the biological circuitry of the human brain. It consists of an input layer, multiple hidden layers of "neurons," and an output layer [\[4\]](#). In the context of language, these models are designed to process sequences of data, learning to understand and generate human-like text.

Why "Language"?

LLMs are specialized for **Natural Language Processing (NLP)**. They perform a wide range of tasks including question answering, translation, sentiment analysis, and text summarization [5]. While traditional NLP models were often "task-specific," modern LLMs are "general-purpose," capable of performing various tasks within a single architecture [6].

The Significance of "Large"

The "Large" in LLM refers to two primary factors: the number of parameters and the volume of training data.

- 1. **Parameters:** These are the internal weights of the neural network that are adjusted during training. While earlier models had significantly fewer parameters, modern LLMs deal in the billions [7].
- 2. **Scale Comparison:**
 - **GPT-2:** ~1.5 billion parameters [8].
 - **GPT-3:** 175 billion parameters [9].
 - **Llama 3.1:** 405 billion parameters [10].

Table 1: Evolution of GPT Parameter Scale

Model Version	Parameter Count
GPT-1	117 Million
GPT-2	1.5 Billion
GPT-3	175 Billion

[11].

The Limitations of RNNs and LSTMs

RNNs and LSTMs were the standard before Transformers, but they faced challenges in capturing long-range dependencies in sentences compared to the newer architecture [12].

"Attention Is All You Need"

In 2017, researchers published the seminal paper "Attention Is All You Need," which introduced the Transformer [\[13\]](#). This architecture introduced a mechanism called **Self-Attention**. Self-Attention allows the model to capture long-range dependencies in a sentence, allowing for a much better job at predicting the next word [\[12\]](#).

Encoder vs. Decoder

The original Transformer architecture consisted of two main parts: 1. **Encoder:** Processes the input text [\[14\]](#). 2. **Decoder:** Generates the output text [\[15\]](#).

Crucially, the GPT (Generative Pre-trained Transformer) lineage of models deviates from this by using **only the decoder block** [\[16\]](#).

Stages of Building an LLM

Building an LLM is a multi-stage process that moves from raw data to a specialized application. These stages are broadly categorized into **Pre-training** and **Fine-tuning** [\[17\]](#).

Stage 1: Data Preparation and Architecture Design

Before training begins, the data must be formatted. This involves: * **Tokenization:** Breaking sentences into smaller units called tokens [\[18\]](#). * **Vector Embedding and Positional Encoding:** Converting tokens into numerical vectors and adding information about the order of words [\[19\]](#).

Stage 2: Pre-training (The Foundational Stage)

Pre-training involves training on a large and diverse data set [\[20\]](#).

- **The Task:** The model is trained on a simple objective: **Next-Word Prediction**. Given a sequence of words, the model must predict the next word [\[21\]](#).
- **The Data:** GPT-3 was trained on a massive corpus, including:
 - **Common Crawl (60%):** A repository of web data [\[22\]](#).
 - **WebText2:** Data consisting of Reddit submissions [\[18\]](#).

- **Books1, Books2, and Wikipedia:** Additional high-quality text sources [\[18\]](#).
- **The Cost:** Pre-training GPT-3 cost an estimated **\$4.6 million** [\[23\]](#).

The result of this stage is a **Foundational Model** [\[24\]](#).

Stage 3: Fine-tuning (The Specialization Stage)

Fine-tuning involves taking the foundational model and training it further on a narrower data set for a specific task or domain [\[25\]](#).

1. **Instruction Fine-tuning:** Training the model to follow commands [\[26\]](#).
2. **Classification Fine-tuning:** Training the model for specific tasks like spam detection or sentiment analysis [\[27\]](#).
3. **Domain Adaptation:** Fine-tuning models on domain-specific data, such as banking or legal records [\[28\]](#).
4. **GPT-2 (2019):** Scaled the model to 1.5 billion parameters [\[29\]](#).
5. **GPT-3 (2020):** Scaled to 175 billion parameters and introduced the concept of few-shot learning [\[30\]](#).

Zero-Shot, One-Shot, and Few-Shot Learning

GPT models have the ability to learn "in-context" [\[31\]](#).

- **Zero-Shot:** Asking the model to perform a task with no examples [\[32\]](#).
- **One-Shot:** Providing one example [\[32\]](#).
- **Few-Shot:** Providing several examples [\[33\]](#).

Emergent Behavior

A phenomenon in LLMs is **Emergent Behavior**, where abilities appear in large models that were not present in smaller ones [\[34\]](#). This includes tasks like coding or logical reasoning [\[35\]](#).

Technical Implementation Overview

Data Pre-processing

1. **Loading the Dataset:** Starting with a large corpus of unlabeled text [\[36\]](#).
2. **Tokenization:** Mapping text to integer IDs.

3. **Sampling:** Creating input-target pairs for next-token prediction [\[37\]](#).

The Training Loop

1. **Forward Pass:** Input tokens pass through the layers [\[38\]](#).
2. **Loss Calculation:** Comparing the prediction to the actual next token.
3. **Backpropagation:** Updating parameters to reduce error [\[39\]](#).
4. **Autoregression:** The model's output is fed back as input to generate the next word [\[40\]](#).

[\[41\]](#)

Modern Context: Open Source vs. Closed Source

The gap between closed-source and open-source models is closing [\[42\]](#).

- **Closed Source:** Models accessed via API where weights are kept secret [\[43\]](#).
- **Open Source:** Models where weights are released to the public, allowing users to run them on their own hardware [\[43\]](#).

Applications

Developers can build: * **Custom Chatbots:** For specific industries [\[44\]](#). * **Content Generators:** Writing articles or poems [\[45\]](#). * **Educational Tools:** Generating lesson plans or worksheets [\[46\]](#). * **Sentiment Analysis:** Analyzing feedback or detecting hate speech [\[47\]](#). * **Machine Translation:** Converting text between languages [\[48\]](#).

Summary

An LLM is a massive neural network utilizing the **Transformer's decoder-only architecture** for **next-word prediction** [\[49\]](#).

This chapter focuses on the very first building block of stage one: the data preprocessing pipeline. Large language models are essentially specialized neural networks. Like all neural networks, they require numerical data as input. However, the raw data available for training—billions of documents—exists as human-readable text. The bridge between raw text and the numerical tensors required by a Transformer architecture is built through tokenization and embedding [\[50\]](#).

Step 1: Tokenization Fundamentals

At its most basic level, tokenization is the process of breaking down a sentence into individual units called tokens. While it may seem as simple as splitting a sentence into words, the implementation for modern LLMs involves significant nuance [\[51\]](#).

Word-Based Tokenization from Scratch

To understand the mechanics, we can implement a basic word-based tokenizer using Python's regular expression library (`re`). For our training data, we utilize a public domain short story, "The Verdict" by Edith Wharton (1908) [\[52\]](#).

The first challenge in tokenization is handling punctuation and white space. A naive approach using `re.split(r'(\s)', text)` would split text only at white spaces, leaving punctuation attached to words. To build a more robust tokenizer, we must explicitly split on punctuation marks such as commas and periods [\[53\]](#).

```
import re

text = "Hello, world. This is--an example."
# Splitting on whitespace and punctuation
result = re.split(r'([.,:;?_!"()\`]|--|\s)', text)
# Removing redundant whitespace tokens
result = [item.strip() for item in result if item.strip()]
print(result)
# Output: ['Hello', ',', 'world', '.', 'This', 'is', '--', 'an', 'example', '.']
```

This simple scheme ensures that punctuation marks are treated as individual tokens, allowing the model to learn their structural significance [\[54\]](#).

Building the Vocabulary

Once the text is broken into tokens, we must map each unique token to a unique integer, known as a token ID. This mapping is stored in a "vocabulary." To construct a vocabulary, we take all tokens from our training set, remove duplicates to find unique tokens, and sort them alphabetically [\[55\]](#).

Table 2: *Vocabulary Mapping Example*

Token	Token ID
,	0
.	1
a	2
apple	3
banana	4

In Python, this is efficiently handled by creating a dictionary where keys are tokens and values are integers. We also require an inverse dictionary (integer-to-string) for the "decoding" process, which converts the model's numerical output back into human-readable text [\[56\]](#).

Handling Unknown Words and Special Tokens

A significant limitation of the word-based tokenizer is the "Out of Vocabulary" (OOV) problem. If a user provides a word not found in the training data, the tokenizer will fail to map it to an existing ID [\[57\]](#).

To solve this, we introduce special context tokens: 1. `<|unk|>` (**Unknown Token**): Any word not in the vocabulary is mapped to this ID. 2. `<|endoftext|>`: This acts as a marker between unrelated documents in a large training set [\[58\]](#).

Other common special tokens include `<|bos|>` (beginning of sequence), `<|eos|>` (end of sequence), and `<|pad|>` (padding for batch processing) [\[59\]](#).

Step 2: Advanced Tokenization - Byte Pair Encoding (BPE)

While word-based tokenization is intuitive, modern LLMs like GPT-2 and GPT-3 use a more sophisticated method called Byte Pair Encoding (BPE) [\[60\]](#).

The Three Paradigms of Tokenization

Table 3: *Tokenization Comparison*

Type	Description	Pros	Cons
Word-based	Every word is a token.	Intuitive.	Large vocabulary, OOV issues.
Character-based	Every character is a token.	Tiny vocabulary, no OOV issues.	Long sequence lengths.
Subword-based (BPE)	Breaks words into frequent chunks.	Balanced vocabulary, handles OOV.	More complex implementation.

BPE is a "best of both worlds" approach. It ensures frequent words are single tokens while rare words are broken into subwords [\[61\]](#).

The BPE Algorithm Mechanics

BPE works by iteratively merging the most frequent pair of consecutive characters or subwords in a dataset [\[62\]](#).

Example Walkthrough: 1. **Initial State:** Split words into characters. 2. **Iteration 1:** Identify the most frequent pair and merge them into a new subword token. 3. **Iteration 2:** Identify the next most frequent pair and merge. 4. **Result:** The vocabulary now contains meaningful subwords or suffixes [\[63\]](#).

This allows the model to understand relationships between words with shared roots, which is often lost in word-based schemes [\[64\]](#).

Implementation with Tiktoken

In practice, we use optimized libraries like OpenAI's `tiktoken`. It handles the complex BPE merges efficiently and includes the `<|endoftext|>` token by default. Notably, BPE tokenizers do not need an `<|unk|>` token because they can always fall back to individual characters to represent any word [\[65\]](#).

```
import tiktoken
tokenizer = tiktoken.get_encoding("gpt2")
text = "Hello, world!"
ids = tokenizer.encode(text)
# Output: [15496, 11, 995, 0]
```

Step 3: Creating Input-Target Pairs

LLMs are trained using a self-supervised learning objective: predicting the next token in a sequence [\[66\]](#).

The Sliding Window Approach

To prepare data for training, we take a sequence of tokens and create two variables: **X** (Input) and **Y** (Target). Y is simply X shifted by one position to the right [\[67\]](#).

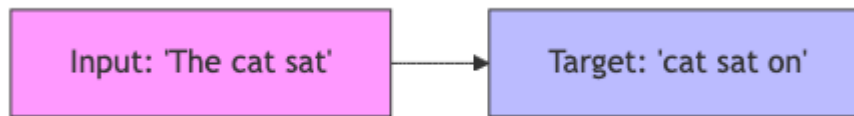


Figure 1: *Sliding Window Prediction*

If our context size is 4, and our sequence is `[1, 2, 3, 4, 5]` : - Input: `[1, 2, 3, 4]` - Target: `[2, 3, 4, 5]`

This single pair represents multiple training examples where the model learns to predict the next token at each step [\[68\]](#).

DataLoaders and Batching

For efficient training, we use PyTorch's `Dataset` and `DataLoader` classes. This allows us to organize data into batches and utilize parallel processing [\[69\]](#).

Key parameters include: - **Batch Size:** How many sequences the model processes before updating weights. - **Context Length:** The maximum number of tokens in an input sequence. - **Stride:** How many tokens the "window" slides forward to create the next batch [\[70\]](#).

Step 4: Token Embeddings

Token IDs are just arbitrary integers. They do not convey semantic meaning. To solve this, we convert IDs into **Token Embeddings**—vectors in a high-dimensional space [\[71\]](#).

Semantic Vector Space

In a well-trained embedding space, words with similar meanings are positioned close to each other. This allows the model to capture semantic relationships, such as the relationship between "dog" and "puppy" [\[72\]](#).

By representing tokens as vectors, the model can exploit the inherent relationships in language [\[73\]](#).

The Embedding Layer as a Lookup Table

In PyTorch, the `torch.nn.Embedding` layer is essentially a large weight matrix. If the vocabulary size is 50,257 and the embedding dimension is 768, the matrix has dimensions `50257 x 768` [\[74\]](#).

When we pass a token ID to this layer, it retrieves the corresponding row of the matrix. These weights are initialized randomly and are optimized during training [\[75\]](#).

Step 5: Positional Embeddings

A major limitation of the Transformer architecture is that it processes all tokens in a sequence simultaneously. Consequently, the model has no inherent sense of word order [\[76\]](#).

Absolute vs. Relative Positioning

To fix this, we add **Positional Embeddings** to the token embeddings. - **Absolute Positional Embeddings:** Each position has a unique vector. This vector is added to the token embedding at that position. GPT models typically use learned absolute positional embeddings [\[77\]](#). - **Relative Positional Embeddings:** Focuses on the distance between tokens rather than their exact index [\[78\]](#).

Implementation and Broadcasting

The positional embedding matrix has dimensions `Context_Length x Embedding_Dimension`. For a batch of data, PyTorch uses "broadcasting" to add these positional vectors to every sequence in the batch [\[79\]](#).

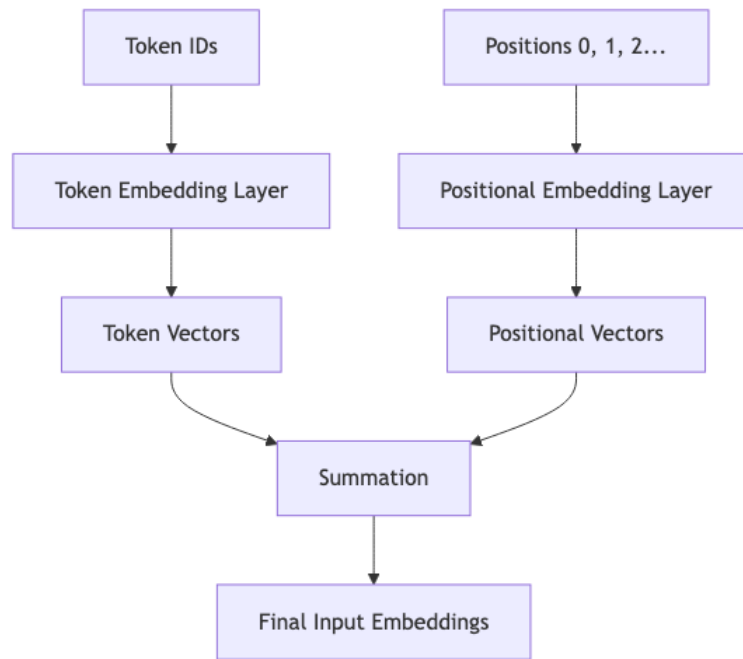


Figure 2: *Final Input Embedding Construction*

The final **Input Embedding** is the sum: `Token_Embedding` + `Positional_Embedding`. This combined vector now contains both the semantic meaning of the word and its structural role in the sentence [\[80\]](#).

Summary: The Complete Preprocessing Pipeline

The journey from raw text to model-ready tensors involves a precise sequence of operations. We begin by cleaning and splitting text into tokens using Byte Pair Encoding, which balances vocabulary size with semantic richness. These tokens are converted into unique IDs, which are then organized into input-target pairs using a sliding window and a PyTorch DataLoader to facilitate batch training [\[81\]](#).

To make these IDs useful for a neural network, we map them to high-dimensional vectors via an embedding lookup table. Finally, we inject spatial awareness by adding positional embeddings. The resulting "Input Embeddings" are the actual data fed into the Transformer, carrying the weight of the text's meaning and structure [\[82\]](#).

The Attention Mechanism

Opening Context: The Engine of the Transformer

In the evolution of natural language processing, the transition from recurrent architectures to the Transformer model marked a paradigm shift. If the Transformer is the vehicle that enables the capabilities of modern Large Language Models (LLMs), the attention mechanism is the engine that drives it [\[83\]](#). Prior to this innovation, sequence modeling relied heavily on Recurrent Neural Networks (RNNs). While these models introduced the concept of "memory" through hidden states, they suffered from fundamental bottlenecks in handling long-range dependencies.

The attention mechanism solves a core problem in linguistics: context. In a sentence like *"The cat that was sitting on the mat which was next to the dog jumped,"* a human reader instinctively knows that the subject "cat" is the entity that performed the action "jumped." However, for a machine, the distance between "cat" and "jumped" creates a "long-term dependency" [\[84\]](#). Without attention, a model might struggle to correctly associate the jumping action with the "cat" because of the complexity of the sentence. Attention allows the model to dynamically focus on relevant parts of the input, effectively creating an "enriched" representation of every word based on its surroundings [\[85\]](#).

This chapter provides a deep technical exploration of building the attention mechanism from scratch. We will progress from the most basic "simplified" self-attention to the complex, multi-headed causal attention used in production-grade LLMs.

The Limitations of Recurrent Architectures

To appreciate the necessity of attention, one must understand the failure modes of the architectures that preceded it. RNNs process data sequentially, one token at a time. As the model moves through a sentence, it maintains a "hidden state"—a vector intended to encapsulate the memory of all previous tokens [\[86\]](#).

The Information Bottleneck

In an RNN-based setup, the encoder processes the input sequence and compresses it into a "final hidden state." This vector is then passed to the

decoder to generate the output [87]. The critical flaw is that the decoder relies on that one final vector to represent the entire meaning of the input [88].

For long sentences, this creates a massive information bottleneck. It is difficult for a single fixed-length vector to retain all the nuances, which leads to a "loss of context" where the model may forget the beginning of a sequence by the time it reaches the end [89].

The Sequential Constraint

RNNs are inherently sequential. To calculate the hidden state at a specific time step, the model must have already completed the calculation for the previous time step. This prevents parallelization during training [90].

The Conceptual Shift: Bahdanau Attention

The first major breakthrough came with the introduction of Bahdanau Attention [Reference: <https://arxiv.org/pdf/1409.0473>]. The core idea was to allow the decoder to "selectively access" different parts of the input sequence at each decoding step [91].

Instead of relying on a single context vector, the decoder calculates an "attention score" for every hidden state in the encoder. This allows the model to have a "Dynamic Focus." For example, the model can look back at the entire input and give the highest weight to the most relevant word, ensuring context is preserved [92].

While Bahdanau attention was originally designed for RNNs, it laid the groundwork for the 2017 paper "Attention Is All You Need," which proposed that attention could replace recurrence entirely [Reference: <https://arxiv.org/pdf/1706.03762>].

Simplified Self-Attention: The Purest Form

Before implementing the complex version used in GPT, we must understand the mathematical foundation of "Self-Attention." In self-attention, the model relates different positions of a single sequence to compute a representation of that same sequence [93].

Step 1: Vector Embeddings

We begin with an input sentence: "Your journey starts with one step." Each word is converted into a vector embedding. While GPT uses high-dimensional spaces (e.g., 768 or 1536), we will use a 3-dimensional space for clarity [\[94\]](#).

Table 4: Example 3D Embeddings

Token	Dim 1	Dim 2	Dim 3
Your	0.43	0.15	0.89
Journey	0.12	0.98	0.45
Starts	0.22	0.88	0.33

Step 2: Calculating Attention Scores via Dot Product

To find how much a token (our query) should attend to another token, we need a measure of similarity. The **dot product** is used for this, quantifying the alignment between two vectors [\[95\]](#).

For a query vector x_q and an input vector x_i , the attention score w is:

$$w_{qi} = x_q \cdot x_i$$

Step 3: Normalization with Softmax

Raw dot products are normalized using the Softmax function to make them interpretable as "weights" that sum to 1 [\[96\]](#).

$$\text{Softmax}(x_i) = (e^{x_i} / \sum_j e^{x_j})$$

Using Softmax helps the model focus by amplifying the highest score and pushing lower scores toward zero [\[97\]](#).

Step 4: Computing the Context Vector

The final context vector z is a weighted sum of the input embeddings based on the attention weights [\[98\]](#). This results in an "enriched" embedding that encodes how a word relates to others in the sentence [\[99\]](#).

Scaled Dot-Product Attention: Key, Query, and Value

The simplified version lacks "trainable weights." To allow the model to learn complex relationships, we introduce three trainable weight matrices: W_Q (Query), W_K (Key), and W_V (Value) [\[100\]](#).

The Analogy

- **Query (q):** What I am looking for.
- **Key (k):** What I have to offer (labels for the tokens).
- **Value (v):** The actual information contained in the token [\[101\]](#).

The Mathematical Flow

1. **Projection:** Multiply the input x by W_Q , W_K , W_V to get Q , K , V .
2. **Scores:** QK^T (Dot product of queries and keys).
3. **Scaling:** Divide scores by $\sqrt{d_k}$ (where d_k is the dimension of the keys).
4. **Weights:** Apply Softmax to the scaled scores.
5. **Output:** Multiply weights by V to get the context vector [\[102\]](#).

Why Scale by $\sqrt{d_k}$?

As the dimensionality of vectors increases, the variance of their dot product increases. Large dot products pushed into a Softmax function result in extremely small gradients, causing "vanishing gradients" [\[103\]](#). Dividing by $\sqrt{d_k}$ ensures stable training [\[104\]](#).

```
import torch
import torch.nn as nn

class SelfAttention(nn.Module):
    def __init__(self, d_in, d_out):
        super().__init__()
        self.W_query = nn.Linear(d_in, d_out, bias=False)
        self.W_key = nn.Linear(d_in, d_out, bias=False)
        self.W_value = nn.Linear(d_in, d_out, bias=False)

    def forward(self, x):
        keys = self.W_key(x)
```

```
queries = self.W_query(x)
values = self.W_value(x)

# Compute scaled dot-product attention
attn_scores = queries @ keys.transpose(-2, -1)
attn_weights = torch.softmax(attn_scores / (keys.shape[-1]**0.5), dim=-1)

context_vec = attn_weights @ values
return context_vec
```

Causal Attention: Preventing Information Leakage

In generative tasks like next-word prediction, the model must not "cheat" by looking at future tokens [\[105\]](#). This is achieved through **Causal Masking**.

The Masking Mechanism

We create a mask and replace the scores for future tokens with negative infinity ($-\infty$) [\[106\]](#). When Softmax is applied, these become 0, zeroing out the attention to future tokens [\[107\]](#).

Dropout in Attention

To prevent overfitting, we often apply Dropout to the attention weights, which randomly zeros out some weights during training [\[108\]](#).

Multi-Head Attention: Parallel Perspectives

Multi-Head Attention (MHA) allows the model to attend to information from different representation subspaces simultaneously [\[109\]](#).

Implementation Strategy 1: The Wrapper Approach

One way to implement MHA is to create multiple independent causal attention heads, run them in parallel, and concatenate their outputs [\[110\]](#). If each head produces a specific dimension context vector, the heads are concatenated into a larger vector [\[111\]](#).

Implementation Strategy 2: Weight Splitting (Optimized)

In production, we use a single large linear layer for Q, K, V and then "split" the resulting tensor into multiple heads [\[112\]](#).

The 4D Tensor Transformation

To process multiple heads in parallel, we move from 3D tensors to 4D tensors $(Batch, Heads, Tokens, Head_Dim)$ [\[113\]](#).

1. **Project:** $X \times W_Q \rightarrow (B, T, d_{out})$.
2. **Reshape and Transpose:** Move to $(B, Heads, T, Head_Dim)$ [\[114\]](#).
3. **Compute Attention:** Perform dot products across all heads and batches simultaneously.
4. **Concatenate:** Transpose back and reshape to (B, T, d_{out}) [\[115\]](#).

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, num_heads, context_len, dropout=0.0):
        super().__init__()
        assert d_out % num_heads == 0, "d_out must be divisible by num_heads"

        self.d_out = d_out
        self.num_heads = num_heads
        self.head_dim = d_out // num_heads

        self.W_query = nn.Linear(d_in, d_out, bias=False)
        self.W_key = nn.Linear(d_in, d_out, bias=False)
        self.W_value = nn.Linear(d_in, d_out, bias=False)
        self.out_proj = nn.Linear(d_out, d_out) # Final projection
        self.dropout = nn.Dropout(dropout)
        self.register_buffer("mask", torch.tril(torch.ones(context_len, context_len)))

    def forward(self, x):
        b, num_tokens, d_in = x.shape

        keys = self.W_key(x).view(b, num_tokens, self.num_heads, self.head_dim)
        queries = self.W_query(x).view(b, num_tokens, self.num_heads, self.head_dim)
        values = self.W_value(x).view(b, num_tokens, self.num_heads, self.head_dim)

        # Transpose to (b, num_heads, num_tokens, head_dim)
        keys = keys.transpose(1, 2)
        queries = queries.transpose(1, 2)
        values = values.transpose(1, 2)
```

```

# Scaled dot-product attention
attn_scores = queries @ keys.transpose(-2, -1)
mask_bool = self.mask[:num_tokens, :num_tokens].bool()
attn_scores.masked_fill_(~mask_bool, -float('inf'))

attn_weights = torch.softmax(attn_scores / (keys.shape[-1]**0.5), dim=-1)
attn_weights = self.dropout(attn_weights)

context_vec = (attn_weights @ values).transpose(1, 2).contiguous()
context_vec = context_vec.view(b, num_tokens, self.d_out)

return self.out_proj(context_vec)

```

Summary

The attention mechanism represents a fundamental departure from sequential processing. Key takeaways include: - **RNNs failed** because they compressed sequences into a single vector, losing context [\[116\]](#). - **Self-Attention** allows tokens to interact, creating enriched context vectors [\[117\]](#). - **Key, Query, and Value** matrices provide learnable parameters to focus on linguistic features [\[100\]](#). - **Causal Masking** ensures autoregressive models only learn from the past [\[118\]](#). - **Multi-Head Attention** enables parallel "perspectives" to capture complex relationships [\[119\]](#).

LLM Architecture & Components

LLM Architecture & Components: A Deep Dive into the Transformer

The transition from theoretical understanding to practical implementation marks a pivotal moment in the journey of building a Large Language Model (LLM). Having established the foundations of tokenization, embedding layers, and the self-attention mechanism, we now turn our attention to the structural framework that houses these components: the Transformer architecture. This architecture is a sophisticated engine designed to process sequences of data with contextual depth.

Building an LLM from scratch is a multi-stage process. The first stage involves laying the foundations, which includes data preparation, sampling, and understanding fundamental building blocks like attention. The second stage focuses on pre-training the model, and the third stage involves fine-tuning the model for specific tasks [\[120\]](#). This chapter focuses on the culmination of the first stage: assembling the architectural components into a functional model.

The Birds-Eye View of LLM Architecture

At its highest level, the architecture of a model like GPT (Generative Pre-trained Transformer) can be viewed as a pipeline that transforms input text into a predicted next word. This process begins with tokenization, where input text is broken down into tokens. These tokens are then converted into vector embeddings, which are combined with positional embeddings [\[121\]](#).

The core of this pipeline is the Transformer block. The primary objective of this block is to process input embeddings through several layers to produce an output used for prediction. While an embedding vector captures the meaning of a token, the Transformer block allows tokens to interact through attention, effectively "attending" to relevant parts of the input [\[122\]](#).

The architecture is designed to be stacked. In the case of the GPT-2 small model, the architecture consists of 12 such Transformer blocks stacked sequentially [\[123\]](#).

The Transformer Block: An Internal Anatomy

To understand how the Transformer functions, we must zoom into the individual block. A standard Transformer block consists of several interconnected layers:

1. **Layer Normalization:** Applied to stabilize training.
2. **Masked Multi-Head Attention:** Where tokens interact and exchange information.
3. **Dropout:** A regularization layer.
4. **Shortcut Connections (Residual Connections):** Paths that allow gradients to flow through the network more easily.
5. **Feed-Forward Neural Network (FFN):** A sub-module that processes token information.
6. **GELU Activation:** A specific non-linear activation function used within the FFN.

A critical design feature of the Transformer block is that it maintains the dimensionality of its input. If the input to a block is a tensor of a specific shape, the output will have the exact same shape. This preservation of dimensionality is what enables the stacking of 12 blocks [\[124\]](#).

Layer Normalization: Ensuring Training Stability

Training deep neural networks can be difficult due to the vanishing gradient problem. As a network grows deeper, the gradients used to update weights during backpropagation can become progressively smaller, leading to stagnant learning [\[125\]](#).

Layer Normalization (LayerNorm) addresses training stability by ensuring that the outputs of a layer have a consistent mean and variance. Specifically, LayerNorm adjusts the outputs to have a mean of zero and a unit variance of one [\[126\]](#).

The Mechanism of Layer Normalization

Layer Normalization operates across the feature dimension, making it independent of the batch size [\[127\]](#).

The mathematical procedure for LayerNorm on a specific layer output x is as follows:

1. **Calculate the Mean (μ):** The average of feature values for a single token.
2. **Calculate the Variance (σ^2):** The average of the squared differences from the mean.
3. **Normalize:** Subtract the mean and divide by the square root of the variance (plus a small epsilon ϵ to prevent division by zero).
4. **Scale and Shift:** Multiply by a trainable parameter (scale) and add another trainable parameter (shift).

The scale and shift parameters are learned during the training process alongside the model's weights [\[128\]](#).

```
import torch
import torch.nn as nn

class LayerNorm(nn.Module):
    def __init__(self, emb_dim):
        super().__init__()
        self.eps = 1e-5
        self.scale = nn.Parameter(torch.ones(emb_dim))
```

```

        self.shift = nn.Parameter(torch.zeros(emb_dim))

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        norm_x = (x - mean) / torch.sqrt(var + self.eps)
        return self.scale
* norm_x + self.shift

```

In GPT-2, LayerNorm is applied within each Transformer block (before the attention and before the FFN) and once more after the final Transformer block [\[129\]](#).

GELU Activation and the Feed-Forward Network

While the attention mechanism allows tokens to interact with each other, the Feed-Forward Network (FFN) allows the model to process token information. The FFN in a Transformer block is a sub-module applied to the sequence [\[130\]](#).

The GELU Activation Function

The Gaussian Error Linear Unit (GELU) is an activation function used in modern LLMs. While ReLU is zero for all negative inputs, GELU is a smooth function that allows for small, non-zero values for negative inputs [\[131\]](#).

The advantages of GELU include its smoothness throughout, which can lead to better optimization properties compared to functions with sharp kinks like ReLU [\[132\]](#).

The approximation used in GPT-2 is: $\text{GELU}(x) \approx 0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$ [\[133\]](#).

FFN Architecture: Expansion and Contraction

The FFN follows an "expansion-contraction" design. It consists of linear layers that project the embedding dimension into a higher-dimensional space before projecting it back down to the original embedding space [\[134\]](#).

This design allows the model to explore a richer representation space for each token before returning the data to the standard dimensionality [\[135\]](#).

Shortcut Connections: The Highway for Gradients

Shortcut connections, also known as skip or residual connections, are a fundamental innovation that allows for the training of deep networks. A shortcut connection adds the input of a layer to its output [\[136\]](#).

This addition ensures that the gradient during backpropagation has a direct path to earlier layers, which helps prevent the gradient from vanishing to zero [\[137\]](#).

Shortcut connections smooth the "loss landscape" of the model. Without them, the loss function can be highly non-convex, making optimization difficult. With shortcut connections, the landscape becomes smoother, allowing the optimization algorithm to find the minimum more reliably [\[138\]](#).

Assembling the GPT Model

With the individual components defined, we can assemble the full GPT architecture. The model is defined by a configuration that specifies its scale. For the GPT-2 small (124M) version, the parameters include:

Table: GPT-2 Small Configuration

Table 5:

Parameter	Value
Vocabulary Size	50,257
Context Length	1,024
Embedding Dimension	768
Number of Heads	12
Number of Layers	12
Dropout Rate	0.1
QKV Bias	False

[\[139\]](#)

The Forward Pass Workflow

The execution of the model follows a precise sequence:

1. **Token and Positional Embedding:** Input tokens are converted to vectors and combined with positional embeddings.
2. **Dropout:** Applied to the initial embeddings.
3. **Transformer Blocks:** The data passes through 12 sequential blocks. Each block performs LayerNorm, Attention, Dropout, Shortcut, LayerNorm, FFN, Dropout, and Shortcut [\[140\]](#).
4. **Final LayerNorm:** A final normalization step after the last Transformer block.
5. **Linear Output Head:** Projects the context vectors into the vocabulary space [\[141\]](#).

The output of this process is a tensor that represents scores for the next token in the vocabulary [\[142\]](#).

Generating Text: From Logits to Tokens

The final stage of the LLM architecture is the generation loop. An LLM generates text one token at a time:

1. **Predict:** The model takes the current sequence and produces an output for the next token.
2. **Extract:** We focus on the output for the last token in the input sequence.
3. **Select:** We apply a selection strategy to determine the next token ID [\[143\]](#).
4. **Append:** The new token ID is added to the input sequence.
5. **Repeat:** The expanded sequence is fed back into the model to predict the subsequent token [\[144\]](#).

This process continues until a maximum number of tokens is reached. If the generated sequence exceeds the fixed context length, the input must be truncated to fit the model's limits [\[145\]](#).

Summary

The architecture of a Large Language Model is an orchestration of specialized components. Layer Normalization and Shortcut Connections provide the stability necessary to train deep stacks of Transformer blocks. The Feed-Forward Network, powered by GELU activation, provides the capacity to process token information, while the Multi-Head Attention mechanism enables

contextual understanding. By maintaining dimensionality throughout the stack and utilizing an iterative generation loop, the model transforms embeddings into coherent text. While the initial outputs of a randomly initialized model are nonsensical, the architectural framework is now complete and ready for pre-training on massive datasets.

Pre-training & Model Evaluation

Context and Objectives

The journey of building a Large Language Model (LLM) from scratch reaches a critical inflection point once the architecture—the layers of self-attention, feed-forward networks, and normalization—is finalized. At this stage, the model is a "blank slate" of randomly initialized weights. While it possesses the structural capacity to process language, it lacks the learned internal representations required to generate coherent text.

This chapter focuses on the transition from architecture to intelligence through the process of pre-training and rigorous evaluation. We will explore how to quantify a model's performance using loss functions, the mechanics of the training loop, and the advanced decoding strategies—such as temperature scaling and top-k sampling—that transform raw probability distributions into human-like prose. Finally, we will bridge the gap between academic implementation and industry-standard performance by learning how to load and integrate pre-trained weights from OpenAI's GPT-2 into our custom-built architecture [\[146\]](#).

Measuring LLM Performance: The Loss Function

In traditional machine learning, loss functions are well-defined: Mean Squared Error (MSE) for regression and Cross-Entropy for classification. For an LLM, the task is "next-token prediction," which is essentially a multi-class classification problem where the number of classes equals the vocabulary size [\[147\]](#).

The Input-Target Relationship

To train an LLM, we must provide it with pairs of inputs and targets. Because LLMs are auto-regressive, the target is simply the input shifted by one token. For an input sequence "Every effort moves," the model is tasked with predicting the next token in the sequence. This results in an input tensor and a target tensor of the same shape. If the batch size is 2 and the context length is 4, the target tensor contains the ground-truth token IDs that the model should have predicted at each step [\[148\]](#).

From Logits to Probabilities

When an input tensor passes through the GPT architecture, the final linear layer produces **logits**. These are raw, unnormalized scores for every token in the vocabulary. To convert these into a format suitable for evaluation, we apply the **Softmax** function. This transformation ensures that all values lie between 0 and 1 and sum to 1, representing a probability distribution over the entire vocabulary [\[149\]](#).

Cross-Entropy and Negative Log-Likelihood

The goal of training is to maximize the probability the model assigns to the correct (target) token. Mathematically, it is more efficient to minimize the **Negative Log-Likelihood (NLL)**. If the model predicts a probability P for the correct token, the loss L is:

$$L = -\log(P)$$

If $P=1$ (perfect prediction), $L=0$. If P is very small, L becomes very large. The total loss for a batch is the mean of the NLL across all tokens in that batch [\[150\]](#). In PyTorch, this is implemented efficiently using `torch.nn.functional.cross_entropy`, which combines the log-softmax and NLL loss into a single, numerically stable operation [\[151\]](#).

Perplexity: An Interpretable Metric

While a cross-entropy loss might be mathematically sound, it is difficult to interpret. **Perplexity** provides a more intuitive measure by calculating the exponent of the loss:

```
Perplexity = e^Loss
```

As the model learns, the perplexity drops, indicating that the model is narrowing down the "search space" for the next token [\[152\]](#).

Pre-training on Real-World Datasets

To move beyond toy examples, we must train on substantial text. For educational purposes, we utilize "The Verdict" by Edith Wharton (1906), a short story containing approximately 20,000 characters and 5,000 tokens when processed via Byte Pair Encoding (BPE) [\[153\]](#).

Data Preparation and the Data Loader

The raw text must be partitioned into training (90%) and validation (10%) sets. This split allows us to monitor if the model is truly learning language patterns or merely memorizing the training data (overfitting) [\[154\]](#).

We use a `DataLoader` to chunk the text into batches. Two critical parameters define this process: 1. **Context Size (max_length)**: The number of tokens the model "sees" at once (e.g., 256). 2. **Stride**: How far the "window" slides for the next sample. Setting the stride equal to the context size ensures the model sees all data without overlapping samples in a single batch [\[155\]](#).

The Training Loop Mechanics

The pre-training loop follows a standard deep learning pattern. Even a "small" GPT-2 model has over 100 million parameters [\[156\]](#).

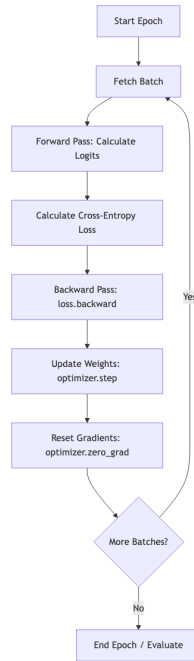


Figure 3: *Training Loop Flow*

The `loss.backward()` call is the engine of pre-training. It propagates the error back through the parameters, calculating the gradient for each [157]. We use the **AdamW** optimizer, a variant of Adam that incorporates weight decay to improve regularization [158].

Monitoring Overfitting

During training, we track both training loss and validation loss. In our experiments with "The Verdict," the training loss dropped significantly, but the validation loss eventually stagnated [159]. This divergence is a hallmark of overfitting, caused by training a high-capacity model on a very small dataset. The model begins to "memorize" specific sentences rather than learning general grammar [160].

Advanced Decoding Strategies

Once a model is trained, we must decide how to pick the next token. Simply picking the token with the highest probability (**Greedy Decoding**) can lead to repetitive text [161].

Temperature Scaling

Temperature scaling adjusts the "sharpness" of the probability distribution by dividing the logits by a factor `T` before applying Softmax:

1. `T = 1` : The original distribution.
2. `T < 1` (**Low Temperature**): The distribution becomes "sharper." The highest probability token gets even more weight, making the model more confident and deterministic.
3. `T > 1` (**High Temperature**): The distribution becomes "flatter" or more uniform. This increases entropy, allowing the model to pick less likely tokens, which fosters variety but increases the risk of nonsensical output [\[162\]](#).

Top-K Sampling

To prevent the model from picking completely nonsensical tokens, we use **Top-K Sampling**. We restrict the model's choices to the `K` most likely tokens. All other tokens are assigned a logit of `-∞`, effectively giving them a 0% probability after Softmax [\[163\]](#).

By combining Top-K and Temperature, we achieve a balance of coherence and variety. For example, setting `K=50` and `T=1.4` often yields more creative results than greedy decoding [\[164\]](#).

Weight Management: Saving and Loading

Training an LLM is computationally expensive. It is vital to save the state of the model to resume training or share it with others.

State Dictionaries

In PyTorch, a model's learnable parameters are stored in a `state_dict` —a Python dictionary mapping each layer name to its parameter tensor [\[156\]](#).

To save a model:

```
torch.save(model.state_dict(), "model.pth")
```

However, saving just the model is often insufficient. Optimizers like AdamW maintain internal states. If you resume training without these, the optimizer

must "re-learn" the landscape [165]. Therefore, we save a **checkpoint** containing both the model and optimizer states:

```
checkpoint = {
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
}
torch.save(checkpoint, "checkpoint.pth")
```

Integrating OpenAI GPT-2 Weights

The ultimate test of our custom GPT architecture is its compatibility with industry-standard weights. OpenAI released GPT-2 weights which we can map to our implementation [166].

The Mapping Challenge

OpenAI's weights are stored in a specific naming convention. We must map these to our internal class attributes.

Table 6: *Weight Mapping for GPT-2*

OpenAI Key	Our Architecture Component	Description
wte	token_embedding	Token Embedding Matrix
wpe	pos_embedding	Positional Embedding Matrix
h.0.attn.c_attn	blocks.0.attn.W_qkv	Fused Query, Key, Value weights
h.0.mlp.c_fc	blocks.0.ff.fc	Feed-forward expansion layer
h.0.ln_1	blocks.0.norm1	First LayerNorm (Scale/Shift)

Fused Weights and Weight Tying

Two technical nuances are critical during this loading process: 1. **Fused QKV:** OpenAI stores the Query, Key, and Value weights as one large "fused" matrix.

We must split this matrix into three equal parts before assigning them to our model's attention layers [\[167\]](#). **2. Weight Tying:** To save memory, GPT-2 uses the same weights for the initial token embedding and the final output linear layer [\[168\]](#).

Verification

Once the weights are loaded, we can run the model on a prompt like "Every effort moves you." Unlike our model trained on a single book, the OpenAI-weighted model produces highly coherent text [\[169\]](#). This confirms that our architecture is mathematically identical to the one used by OpenAI.

Summary

This chapter has detailed the process of transforming a structural architecture into a functional language model. We established that **Cross-Entropy Loss** and **Perplexity** are the primary yardsticks for measuring an LLM's performance. We implemented a **Pre-training Loop** using the **AdamW** optimizer, observing how high-capacity models can easily overfit small datasets.

To refine the model's output, we explored **Temperature Scaling** and **Top-K Sampling**, which allow us to tune the balance between deterministic logic and creative variation. Finally, we demonstrated the portability of our architecture by successfully loading **OpenAI GPT-2 weights**, proving that our implementation is compatible with state-of-the-art pre-trained parameters.

Classification Fine-tuning

Context and Fundamentals of Fine-tuning

Building a large language model (LLM) from scratch typically involves three distinct stages. Stage one focuses on the architecture, including the attention mechanism, data preparation, and sampling. Stage two involves pre-training the model on massive datasets to learn general language patterns and loading pre-trained weights. However, a pre-trained model, while capable of generating coherent text, is often insufficient for specific industrial or academic applications [\[170\]](#).

Fine-tuning is the process of adapting a pre-trained model to a specific task by training it again on additional data. This is necessary because pre-trained models are trained on general data from the internet and may not reflect the specific requirements or private data of a particular organization [\[171\]](#). For instance, an airline might require a chatbot that understands its specific data, or an educational company might want to build an app using its own proprietary information [\[172\]](#).

The Mechanics of Adaptation

The formal definition of fine-tuning involves the additional training of a pre-existing model that has already acquired features from an extensive dataset. By using a smaller, domain-specific dataset, the model adapts its parameters—specifically its weights and biases—to the new data [\[173\]](#).

There are two primary categories of fine-tuning: 1. **Instruction Fine-tuning:** The model is trained to follow specific instructions or commands [\[174\]](#). 2. **Classification Fine-tuning:** The model is adapted to categorize input into specific categories (e.g., Spam vs. Not Spam) [\[175\]](#).

While instruction fine-tuning is common for general-purpose assistants, this chapter focuses on the implementation of a classification fine-tuning pipeline using the GPT-2 architecture.

Data Preparation and Pre-processing

The foundation of any fine-tuning project is the dataset. For this implementation, we utilize the SMS Spam Collection dataset from the UCI Irvine Machine Learning Repository [\[176\]](#). This dataset contains messages labeled as "ham" (legitimate) or "spam."

Handling Class Imbalance

A critical issue in the UCI dataset is class imbalance: there are 4,825 ham messages but only 747 spam messages [\[177\]](#). Training a model on imbalanced data can lead to biased results. To mitigate this, we perform subsampling to create a balanced dataset containing 747 instances of each class [\[178\]](#).

Label Encoding and Data Splitting

Machine learning models require numerical inputs. We map the text labels to integers: "ham" becomes 0 and "spam" becomes 1 [\[179\]](#). Following standard

practices, the data is split into three subsets: * **Training (70%)**: Used to update model weights. * **Validation (10%)**: Used to monitor performance during training. * **Testing (20%)**: Used for the final evaluation of the model [\[180\]](#).

Implementing the Data Pipeline

To feed data into the LLM efficiently, we implement a custom PyTorch `Dataset` and `DataLoader`. This pipeline handles tokenization, padding, and batching.

Tokenization and the Padding Problem

LLMs process text as sequences of token IDs. We use the Byte Pair Encoding (BPE) tokenizer [\[181\]](#). A significant challenge in batching text is that messages vary in length, but a tensor batch requires every row to have the same number of columns [\[182\]](#).

We adopt a padding approach using the `<|endoftext|>` token (ID 50256) [\[183\]](#). This token signifies the end of a document and ensures all sequences in a batch have a uniform length [\[184\]](#).

The SpamDataset Class

The custom `SpamDataset` class handles encoding every text message into token IDs and ensures sequences match a maximum length, often determined by the longest message in the dataset [\[185\]](#).

```
# Conceptual implementation of the Dataset
class SpamDataset(Dataset):
    def __init__(self, csv_file, tokenizer, max_length=None, pad_token_id=50256):
        self.data = pd.read_csv(csv_file)
        self.encoded_texts = [tokenizer.encode(text) for text in self.data["text"]]

        if max_length is None:
            self.max_length = max(len(t) for t in self.encoded_texts)
        else:
            self.max_length = max_length

        # Truncate if longer than max_length
        self.encoded_texts = [t[:self.max_length] for t in self.encoded_texts]

        # Pad if shorter than max_length
        self.encoded_texts = [
```

```

        t + [pad_token_id]
    * (self.max_length - len(t))
    for t in self.encoded_texts
]

```

[\[186\]](#)

Batching with DataLoaders

The `DataLoader` organizes the dataset into batches (e.g., batch size of 8). This allows for parallel processing and easier data management [\[187\]](#). For our training set, the loader produces approximately 130 batches [\[188\]](#).

Model Architecture and Weight Loading

For classification, we utilize the GPT-2 Small architecture (124 million parameters). The model configuration includes a vocabulary size of 50,257 and an embedding dimension of 768 [\[189\]](#).

Loading Pre-trained Weights

Instead of training from scratch, we load pre-trained weights. This provides the model with a foundational understanding of language [\[190\]](#). We use a loading function to map the parameters into our model instance [\[191\]](#).

The Classification Head

The standard GPT-2 architecture ends with a linear layer that maps to the vocabulary size (50,257). For classification, we replace this "output head" with a new linear layer that maps the embedding dimension (768) to the number of classes (2 for spam/ham) [\[192\]](#).

Parameter Freezing Strategy

To balance efficiency and performance, we freeze the majority of the model and only unfreeze: 1. The new classification head. 2. The final transformer block. 3. The final layer normalization module [\[193\]](#).

This ensures the model retains its general knowledge while specializing its final layers for the classification task [\[194\]](#).

The Classification Mechanism

A unique aspect of using a decoder-only transformer for classification is determining which output token to use for the prediction.

The Last-Token Advantage

Because GPT-2 uses causal self-attention, only the final token has "attended" to all previous tokens in the sequence [\[195\]](#). Consequently, the last token's output contains the most comprehensive representation of the entire message. We extract the last token's output for the classification decision [\[196\]](#).

Logits to Predictions

The output of the classification head consists of "logits." To convert these into a prediction, we identify the index of the highest value (argmax). If index 1 is higher, the prediction is "Spam" [\[197\]](#).

Training and Loss Optimization

Cross-Entropy Loss

We use Cross-Entropy as the loss function. If the model predicts a high probability for the correct label, the loss is low; if it predicts a low probability, the loss is high [\[198\]](#). This function is differentiable, allowing for backpropagation [\[199\]](#).

The Training Loop

The implementation uses the `AdamW` optimizer [\[200\]](#). The training loop follows these steps: 1. Iterate through batches from the `DataLoader`. 2. Perform a forward pass to get logits. 3. Calculate loss against the target labels. 4. Perform a backward pass to compute gradients. 5. Update weights using the optimizer [\[201\]](#).

Performance Monitoring

During training, we monitor both training loss and validation loss. A downward slope indicates the model is learning [\[202\]](#). If training loss drops while validation loss rises, it is a sign of overfitting [\[203\]](#).

Table 7: *Training Progress Example*

Metric	Initial (Pre-train)	Final (Fine-tuned)
Training Accuracy	~46%	~97%
Validation Accuracy	~45%	~97%
Test Accuracy	~48%	~95%
Training Loss	~0.70	~0.08

[\[204\]](#)

Inference and Model Deployment

The Inference Pipeline

To classify a new message, we replicate the pre-processing steps: tokenize the input, pad to the required length, pass it through the model, and extract the logits from the last token to get the final label [\[205\]](#).

Practical Examples

A fine-tuned model identifies spam patterns, such as messages about winning cash, while correctly identifying personal messages as ham [\[206\]](#).

Saving the Model

The model's weights can be saved using `torch.save()`, allowing the model to be reloaded later for production use [\[207\]](#).

Instruction Fine-tuning & Conclusion

1. Context: The Transition from Pre-training to Instruction Following

The journey of building a Large Language Model (LLM) from scratch typically follows a rigorous three-stage progression. In the initial stages, we focused on data preparation, the mathematical intricacies of the multi-head attention mechanism, and the structural assembly of the Transformer architecture. This was followed by the pre-training phase, where the model learned to predict the next token across a massive corpus of text, effectively acquiring a foundational understanding of language [\[208\]](#).

However, a pre-trained "foundation" model is primarily a text completion engine. If prompted with an instruction, a base model might simply repeat the input or continue the text in a non-responsive way because it interprets the input as the start of a sequence rather than a direct command [\[209\]](#). To transform this completion engine into a useful personal assistant, we must employ **Instruction Fine-tuning**.

Instruction fine-tuning is a form of supervised learning where the model is specifically trained to follow human commands. This chapter details the process of preparing instruction datasets, formatting them into specialized prompt templates, batching sequences, and executing the fine-tuning loop to create a responsive, command-following agent.

2. The Rationale for Instruction Fine-tuning

Pre-trained models struggle with specific tasks like grammar correction or active-to-passive voice conversion without explicit fine-tuning [\[210\]](#). There are two primary drivers for this process:

2.1 Command Adherence

A base model can generate fluent sentences but lacks the alignment to treat a prompt as a task. For instance, if asked to "fix the grammar in this text," a base model might simply continue the text instead of editing it. Fine-tuning

provides the model with the behavioral blueprint to recognize an instruction and provide a corresponding response [\[211\]](#).

2.2 Domain Specificity

In corporate environments, models often need to understand private policies. An e-commerce company might fine-tune a model on its specific return policies to ensure the chatbot provides accurate, company-specific information rather than generic guesses [\[212\]](#). Similarly, in healthcare, fine-tuning allows an assistant to follow specific medical guidelines that were not present in the general pre-training data [\[213\]](#).

3. Dataset Preparation and Formatting

The success of instruction fine-tuning depends on the quality and structure of the training data. For this implementation, we utilize a dataset consisting of approximately 1,100 instruction-response pairs [\[214\]](#).

3.1 Data Structure

Each entry in the dataset typically contains three keys: 1. **Instruction:** The task description. 2. **Input:** The context or data the task acts upon. 3. **Output:** The desired ground-truth response.

Some entries may lack an "Input" field if the instruction is self-contained, such as "What is the capital of Indonesia?" [\[215\]](#).

3.2 The Alpaca Prompt Format

Models perform better when instructions are wrapped in a consistent "prompt template." One of the most widely adopted formats is the **Stanford Alpaca** style [\[216\]](#). This format structures the data into a human-readable block that clearly delineates the task from the context.

The template used for fine-tuning is as follows:

"Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request."

In the Alpaca format, if the input is missing, the template is modified to remove the "paired with an input" clause [\[217\]](#).


```
def format_input(entry):
    instruction_text = (
        f"Below is an instruction that describes a task. "
        f"Write a response that appropriately completes the request.\n\n"
        f"
    )
    ### Instruction:\n{entry['instruction']}
    )
    input_text = f"\n\n
    ### Input:\n{entry['input']}" if entry.get('input') else ""
    return instruction_text + input_text
```

4. Advanced Data Batching and Tokenization

Batching instruction data is complex because instruction-response pairs vary in length. We must implement a custom pipeline to ensure all sequences in a batch are computationally compatible.

4.1 The Tokenization Workflow

The workflow involves: 1. Formatting the raw JSON entry into the Alpaca string. 2. Appending the ground-truth response to the string. 3. Converting the entire string into a list of token IDs.

4.2 Padding and the End-of-Text (EOT) Token

Because sequences in a batch have different lengths, we must "pad" shorter sequences to match the longest sequence in that specific batch [\[218\]](#). We use the token ID `50256`, which represents the `<|endoftext|>` token in GPT-2 [\[219\]](#).

4.3 Creating Target IDs (The Shift-Right Mechanism)

The "Target" tensor is a copy of the "Input" tensor shifted to the right by one position [\[220\]](#). This forces the model to learn that given the instruction and input, the next logical tokens should be the response.

4.4 The Ignore Index (-100)

We replace padding tokens in the **Target** tensor with the value `-100` [\[221\]](#). PyTorch's loss function ignores any target with the value `-100`, ensuring the model is not penalized for predicting padding tokens [\[222\]](#).

5. Loading Pre-trained Weights

Rather than initializing the model with random weights, we load pre-trained weights from GPT-2. For this instruction fine-tuning example, we utilize the **GPT-2 Medium (355M parameters)** model [\[223\]](#).

5.1 Model Configuration

The GPT-2 Medium configuration includes: - **Vocabulary Size:** 50,257 - **Context Length:** 1,024 tokens - **Transformer Blocks:** 24 - **Attention Heads:** 16 - **Embedding Dimension:** 1,024 [\[224\]](#)

5.2 Weight Mapping

We use a utility function to map the downloaded checkpoints into our custom PyTorch architecture. This involves populating the weight matrices for the query, key, and value projections, as well as the feed-forward layers and layer normalization parameters [\[225\]](#).

6. The Fine-tuning Loop

The fine-tuning process utilizes the **AdamW** optimizer to minimize the Cross-Entropy loss [\[226\]](#).

6.1 Hyperparameters

- **Epochs:** 1 to 2 (depending on compute availability)
- **Batch Size:** 8 [\[227\]](#)

6.2 Training Dynamics

A successful fine-tuning run shows a decrease in loss as the model aligns its pre-trained knowledge with the instruction format [\[228\]](#).

6.3 Instruction Masking (Research Perspective)

There is a debate regarding whether the loss should be calculated over the *entire* prompt or only the *response* part. - **Standard Approach:** Calculate loss over the whole sequence (Instruction + Input + Response). - **Masking Approach:** Replace Instruction and Input tokens in the Target tensor with `-100`, so the model only learns from the Response [\[229\]](#).

Some research suggests that *not* masking may actually benefit model performance by providing more signal during training [\[230\]](#).

7. Evaluation Methodologies

Evaluating an instruction-tuned model is non-trivial because "correctness" is often qualitative. We employ three primary strategies:

7.1 MMLU (Massive Multitask Language Understanding)

MMLU is a benchmark consisting of 57 tasks across STEM, the humanities, and social sciences [\[231\]](#).

7.2 Human Preference

The gold standard remains human evaluation, where annotators rank model responses based on helpfulness, honesty, and harmlessness [\[232\]](#).

7.3 LLM-as-a-Judge (Ollama and Llama-3)

A modern approach involves using a much larger model (like **Llama-3 8B**) to grade the responses of our smaller model [\[233\]](#). We provide the judge model with: 1. The original instruction. 2. The ground-truth output. 3. Our model's generated response.

The judge model then assigns a score and provides a qualitative justification [\[234\]](#).

8. Practical Implementation with Ollama

To facilitate automated evaluation, we use **Ollama**, a tool for running LLM inference locally [\[235\]](#).

8.1 Setup

1. Download Ollama.
2. Run `ollama run llama3` in the terminal to pull the 8B parameter model.
3. Use the API to send prompts to Llama-3 and retrieve scores for our fine-tuned GPT-2 model [\[236\]](#).

8.2 Example Evaluation Result

- **Instruction:** "Name the author of Pride and Prejudice."
- **Ground Truth:** "Jane Austen."
- **Model Response:** "George Bernard Shaw."
- **Llama-3 Score:** 0/100.
- **Justification:** The response is factually incorrect as George Bernard Shaw was not the author of Pride and Prejudice [\[237\]](#).

9. Improving Model Performance

If the average evaluation score is low, several levers can be pulled: 1. **Increase Epochs:** Moving from 1 to 2 epochs often improves results [\[238\]](#). 2. **Larger Datasets:** Transitioning from 1,100 pairs to the full 52,000 pairs in the Stanford Alpaca dataset significantly improves generalization [\[239\]](#). 3. **PEFT (Parameter-Efficient Fine-Tuning):** Techniques like **LoRA** allow for faster training by only updating a small subset of weights [\[240\]](#).

10. Summary and Conclusion

We have navigated the transition from a generic pre-trained model to a specialized instruction-following assistant. By implementing a custom data pipeline—encompassing Alpaca formatting, dynamic padding, and target shifting—we provided the necessary structure for the model to learn task-oriented behavior.

The fine-tuning loop, powered by the AdamW optimizer and the GPT-2 Medium architecture, demonstrated that even a modest number of parameters

can achieve alignment with human instructions. Finally, by leveraging Ollama and Llama-3 as automated judges, we established a quantitative framework for measuring progress.

This concludes the "Build LLMs from Scratch" series. You now possess the foundational knowledge to architect, train, and align your own models from the ground up [\[241\]](#).

Conclusion

By completing this journey, you have transitioned from a conceptual understanding of artificial intelligence to a rigorous, ground-up mastery of Large Language Models. You began by deconstructing the raw building blocks of language through tokenization and data preprocessing, eventually breathing life into those numbers through the implementation of the multi-head attention mechanism. By assembling these components into a complete transformer architecture and navigating the complexities of pre-training and evaluation, you have gained a rare perspective on the internal mechanics that govern today's most sophisticated AI systems. You have not merely used a model; you have engineered one, transforming a mathematical blueprint into a functional engine capable of predicting the next token in a sequence.

The final stages of our exploration demonstrated that a pre-trained model is only the beginning of the story. Through the lenses of classification and instruction fine-tuning, you learned how to steer a general-purpose engine toward specific, human-centric tasks, bridging the gap between raw statistical probability and meaningful interaction. As you close this book, you possess a foundational toolkit that allows you to demystify new research papers and adapt to an ever-evolving field. Your next steps might involve exploring parameter-efficient fine-tuning techniques like LoRA, diving into the world of Reinforcement Learning from Human Feedback (RLHF), or scaling your architectures to handle multimodal inputs. The black box has been opened, and the path forward is now yours to define.

Glossary

Absolute Positional Embeddings — A method where each position in a sequence has a unique vector that is added to the token embedding to provide the model with information about word order.

Autoregression — A process where the model's own output is fed back as input to generate the subsequent word in a sequence.

Backpropagation — The process of updating a neural network's parameters to reduce the error between its predictions and the actual targets.

Batch Size — The number of sequences a model processes before updating its internal weights during training.

Byte Pair Encoding (BPE) — A subword-based tokenization method that iteratively merges the most frequent pairs of characters or subwords to balance vocabulary size and handle unknown words.

Classification Fine-tuning — The process of training a model for specific categorization tasks such as spam detection or sentiment analysis.

Closed Source — Models accessed via API where the internal weights are kept secret from the public.

Context Length — The maximum number of tokens allowed in a single input sequence for the model.

Decoder — The part of the original Transformer architecture responsible for generating output text.

Deep Neural Network — A computational structure inspired by biological circuitry consisting of an input layer, multiple hidden layers of neurons, and an output layer.

Domain Adaptation — Fine-tuning a model on data specific to a particular field, such as legal or banking records.

Emergent Behavior — The phenomenon where complex abilities like coding or logical reasoning appear in large models despite not being present in smaller versions.

Encoder — The part of the original Transformer architecture designed to process input text.

Few-Shot Learning — A task where the model is provided with several examples to learn how to perform a specific function in-context.

Fine-tuning — The stage of development where a foundational model is trained further on a narrower dataset for a specific task or domain.

Forward Pass — The step in the training loop where input tokens pass through the layers of the neural network to produce a prediction.

Foundational Model — The result of the pre-training stage, representing a model trained on a large and diverse dataset before specialization.

Instruction Fine-tuning — Training a model specifically to follow commands and user instructions.

Large Language Model (LLM) — A massive neural network utilizing the Transformer's decoder-only architecture designed to understand, generate, and respond to human-like text.

Loss Calculation — The step in training where the model's prediction is compared to the actual next token to determine the error.

Natural Language Processing (NLP) — A field of artificial intelligence focused on tasks involving human language, such as translation, summarization, and sentiment analysis.

Next-Word Prediction — The primary training objective where a model learns to predict the next word in a sequence given the preceding words.

One-Shot Learning — A task where the model is provided with exactly one example to understand how to perform a function.

Open Source — Models whose weights are released to the public, allowing users to run them on their own hardware.

Parameters — The internal weights of a neural network that are adjusted during the training process.

Positional Embeddings — Vectors added to token embeddings to provide the model with information about the order of words in a sequence.

Pre-training — The foundational stage of building an LLM by training it on a large, diverse dataset using a next-word prediction objective.

Relative Positional Embeddings — A positioning method that focuses on the distance between tokens rather than their exact index in a sequence.

Self-Attention — A mechanism within the Transformer architecture that allows the model to capture long-range dependencies in a sentence for better prediction.

Stride — The number of tokens the sliding window moves forward to create the next batch of training data.

Token Embeddings — Numerical vectors in a high-dimensional space that represent tokens and capture their semantic meanings.

Token ID — A unique integer mapped to a specific token within a model's vocabulary.

Tokenization — The process of breaking down text into smaller units called tokens, such as words or subwords.

Transformer architecture — A generative architecture introduced in 2017 that utilizes self-attention mechanisms to process sequences of data.

Vocabulary — A collection or dictionary that maps unique tokens to their corresponding unique integer IDs.

Zero-Shot Learning — Asking a model to perform a task without providing any prior examples.

References & Resources

Introduction & Fundamentals

1. <https://openai.com/index/language-unsupervised/>
2. <https://arxiv.org/pdf/2005.14165>
3. https://www.business-standard.com/world-news/jpmorgan-chase-unveils-ai-powered-llm-suite-may-replace-research-analysts-124072600460_1.html
4. <https://openai.com/index/introducing-improvements-to-the-fine-tuning-api-and-expanding-our-custom-models-program/>

Data Preprocessing & Tokenization

1. https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
2. https://colab.research.google.com/drive/1ceJDQUHBasbaTB74vg2Fz2N_O2h6C3T?usp=sharing
3. https://drive.google.com/file/d/1ukW7sNpd_4UxI6wKx27Q4fjX3pe6vzta/view?usp=sharing
4. <https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>
5. <https://github.com/openai/tiktoken>

6. https://drive.google.com/file/d/1tF5aWLCZITuWy-1swK3_-63H2NCiYFWH/view?usp=sharing
7. <https://pytorch.org/docs/stable/data.html>
8. https://drive.google.com/file/d/1WxQo6_pG2wwBzxXErgBBPGG2vNt7XHUV/view?usp=sharing
9. https://github.com/rasbt/LLMs-from-scratch/blob/main/ch02/01_main-chapter-code/the-verdict.txt
10. <https://drive.google.com/file/d/1srG6CyModWArPVAumNIJCFDskFrC1COn/view?usp=sharing>
11. https://drive.google.com/file/d/1_lgiXKLwsQEgot4ziei9cWf9Zl91HXcH/view?usp=sharing
12. https://drive.google.com/file/d/1XKi9vE4_nr3B157LjgSx4iu6qUoOSgmB/view?usp=sharing
13. <https://docs.python.org/3/library/re.html>

The Attention Mechanism

1. <https://arxiv.org/pdf/1409.0473>
2. <https://pytorch.org/docs/stable/generated/torch.nn.Linear.html>
3. https://drive.google.com/file/d/1MJAE5wgH76ONzb8W_687ViYyUc7stNWN/view?usp=sharing
4. <https://pytorch.org/docs/stable/generated/torch.nn.Dropout.html>
5. <https://jalammr.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>
6. <https://pytorch.org/docs/stable/generated/torch.nn.Module.html>
7. <https://drive.google.com/file/d/1b5b2PG55PjSYWkPhHy4Q3mQwjBXmCOTX/view?usp=sharing>
8. https://drive.google.com/file/d/1I2RXY24kWhQ8ur3WXLjnQD2n9_C6Fj42/view?usp=sharing
9. <https://arxiv.org/pdf/1706.03762>
10. <https://pytorch.org/docs/stable/generated/torch.tril.html>
11. <https://pytorch.org/docs/stable/generated/torch.triu.html>
12. <https://drive.google.com/file/d/17-GbtuioC4fnn9AhLR2m4oSgcpTNzIbu/view?usp=sharing>
13. <https://drive.google.com/file/d/13oYPEzurWAV4AbnFV9JWAG4uJVM65ZWB/view?usp=sharing>
14. https://pytorch.org/docs/stable/generated/torch.Tensor.masked_fill.html#torch.Tensor.masked_fill

15. <https://discuss.pytorch.org/t/what-is-the-difference-between-register-buffer-and-register-parameter-of-nn-module/32723>
16. <https://pytorch.org/docs/stable/generated/torch.nn.Softmax.html>

LLM Architecture & Components

1. <https://drive.google.com/file/d/1k4TwMW6HHDiS1tcbGO5ip3D3BonwSao/view?usp=sharing>

Pre-training & Model Evaluation

1. https://drive.google.com/file/d/1H6E79zXNsdG_Hn8y2wh6p6mj8o5GGJmD/view?usp=sharing
2. https://drive.google.com/file/d/1JX_DNJ9Xvty_ItdHhl_3TyOfahkGvvI/view?usp=sharing
3. <https://www.kaggle.com/datasets/xhlulu/openai-gpt2-weights>
4. https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
5. https://pytorch.org/docs/stable/generated/torch.nn.functional.cross_entropy.html
6. https://pytorch.org/tutorials/recipes/recipes/what_is_state_dict.html#what-is-a-state-dict-in-pytorch
7. <https://drive.google.com/file/d/1mH02cCTT0575ecaNnaDGAm2TTXvpXU3t/view?usp=sharing>
8. https://pytorch.org/docs/stable/generated/torch.optim.Optimizer.state_dict.html
9. <https://drive.google.com/file/d/1p87nemE6wFjGhkX4kC7KfGv1juMZJrMr/view?usp=sharing>
10. <https://pytorch.org/docs/stable/generated/torch.optim.AdamW.html>
11. <https://drive.google.com/file/d/1Uo3f--JSoHe7zbUqcG5qHTBcZMa5YlM3/view?usp=sharing>
12. <https://github.com/openai/tiktoken>
13. <https://medium.com/@harshit158/softmax-temperature-5492e4007f71>
14. https://github.com/rasbt/LLMs-from-scratch/blob/main/ch02/01_main-chapter-code/the-verdict.txt
15. <https://pytorch.org/docs/stable/generated/torch.multinomial.html>
16. https://pytorch.org/tutorials/beginner/saving_loading_models.html
17. <https://drive.google.com/file/d/1k--iNpyoynmiHCyn4YC8yiZFbzDNT-nO/view?usp=sharing>

18. <https://pytorch.org/docs/stable/generated/torch.topk.html>

Classification Fine-tuning

1. https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
2. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html
3. https://pytorch.org/docs/stable/generated/torch.nn.functional.cross_entropy.html
4. <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>
5. <https://drive.google.com/file/d/1oXbvoT9t74pEXejQ3o7w9ZABGLjHmFsu/view?usp=sharing>
6. <https://pytorch.org/docs/stable/generated/torch.optim.AdamW.html>
7. <https://pytorch.org/docs/stable/generated/torch.save.html>
8. <https://github.com/openai/tiktoken>
9. <https://drive.google.com/file/d/1XcmKqTul6id86nUUvFsCgy-uBWttvWXf/view?usp=sharing>

Instruction Fine-tuning & Conclusion

1. <https://drive.google.com/file/d/1qyydJT5PCoeK4BopXlbJ3N1f3gPKA6i6/view?usp=sharing>
2. https://drive.google.com/file/d/1FQIzAGdBVgSqR_39Qx4jQY0AOYARkkld/view?usp=sharing
3. https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
4. <https://drive.google.com/file/d/13jgqjSY-GOKhLoOsedUuZPXHTzQXrnt1/view?usp=sharing>
5. https://drive.google.com/file/d/1Gp_f7SmzDBMuzIxSpnR948F1KHKApHM1/view?usp=sharing
6. <https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>
7. https://tatsu-lab.github.io/alpaca_eval/
8. https://github.com/rasbt/LLMs-from-scratch/blob/main/ch07/01_main-chapter-code/instruction-data.json
9. <https://github.com/openai/tiktoken>
10. <https://arxiv.org/abs/2405.14394>
11. <https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>
12. <https://arxiv.org/pdf/2009.03300>

13. https://drive.google.com/file/d/1OCpr4GZVmNNEXDpLuG2GZ1LlnJ_uvqPN/view?usp=sharing
14. https://github.com/tatsu-lab/stanford_alpaca

Footnotes

1. Video: *Series introduction* @ 01:21
2. Video: *Series introduction* @ 04:03
3. Video: *LLM Basics* @ 02:44
4. Video: *LLM Basics* @ 03:24
5. Video: *LLM Basics* @ 11:37
6. Video: *LLM Basics* @ 12:18
7. Video: *LLM Basics* @ 09:35
8. Video: *LLM Basics* @ 08:56
9. Video: *LLM Basics* @ 08:15
10. Video: *How does GPT-3 really work?* @ 25:17
11. Video: *LLM Basics* @ 08:15; 09:35]

The Secret Sauce: The Transformer Architecture

*The primary catalyst for the current AI boom was the introduction of the **Transformer architecture** in 2017. Before Transformers, the field relied on Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks [Video: "How does GPT-3 really work?" @ 33:52*

12. Video: *How does GPT-3 really work?* @ 33:52
13. Video: *What are transformers?* @ 02:44
14. Video: *How does GPT-3 really work?* @ 20:08
15. Video: *How does GPT-3 really work?* @ 20:50
16. Video: *How does GPT-3 really work?* @ 25:59
17. Video: *Stages of building an LLM* @ 01:25
18. Video: *Stages of building an LLM* @ 04:48
19. Video: *Stages of building an LLM* @ 05:30
20. Video: *Stages of building an LLM* @ 02:07
21. Video: *Stages of building an LLM* @ 06:14
22. Video: *Stages of building an LLM* @ 04:09
23. Video: *Stages of building an LLM* @ 18:37
24. Video: *Stages of building an LLM* @ 19:18
25. Video: *Stages of building an LLM* @ 11:40
26. Video: *Stages of building an LLM* @ 24:06
27. Video: *Stages of building an LLM* @ 25:29
28. Video: *Stages of building an LLM* @ 14:29; 16:32].

Understanding GPT: Generative Pre-trained Transformers

The GPT series represents a successful implementation of the decoder-only Transformer architecture.

The Evolution of GPT

- **GPT-1 (2018):** Introduced generative pre-training on unlabeled text [Video: "How does GPT-3 really work?" @ 02:45]
- Video: *How does GPT-3 really work?* @ 06:15
- Video: *How does GPT-3 really work?* @ 07:34
- Video: *How does GPT-3 really work?* @ 10:13
- Video: *How does GPT-3 really work?* @ 10:55
- Video: *How does GPT-3 really work?* @ 11:37
- Video: *How does GPT-3 really work?* @ 38:22
- Video: *How does GPT-3 really work?* @ 40:28
- Video: *Stages of building an LLM* @ 22:02
- Video: *Stages of building an LLM* @ 06:11
- Video: *How does GPT-3 really work?* @ 34:56
- Video: *How does GPT-3 really work?* @ 30:49
- Video: *How does GPT-3 really work?* @ 32:51
- Video: *How does GPT-3 really work?* @ 35:36
- Video: *Series introduction* @ 07:28
- Video: *Series introduction* @ 06:47
- Video: *LLM Basics* @ 27:28
- Video: *LLM Basics* @ 26:06
- Video: *LLM Basics* @ 30:14
- Video: *LLM Basics* @ 29:33
- Video: *LLM Basics* @ 28:10

- Video: *Stages of building an LLM* @ 17:20; 18:02]. The process involves **Data Preparation**, **Pre-training** to create a foundational model, and **Fine-tuning** for specialization [Video: *"Stages of building an LLM"* @ 07:35; 09:39].
-

Data Preprocessing & Tokenization

Context: The Foundation of LLM Data Pipelines

Building a Large Language Model (LLM) from scratch is a multi-stage endeavor. The process typically begins with stage one: understanding the core mechanisms, which include data preparation, attention mechanisms, and architecture design. Stage two involves the pre-training of a foundational model through training loops and evaluation. Finally, stage three focuses on fine-tuning the model for specific applications like classification or personal assistants [Video: "Lecture 7 @ 01:23

- Video: *Lecture 7* @ 03:26
- Video: *Lecture 7* @ 00:41
- Video: *Lecture 7* @ 07:31
- Video: *Lecture 7* @ 16:21
- Video: *Lecture 7* @ 20:29
- Video: *Lecture 7* @ 25:20
- Video: *Lecture 7* @ 31:28
- Video: *Lecture 7* @ 41:50
- Video: *Lecture 7* @ 47:12
- Video: *Lecture 7* @ 58:51
- Video: *Lecture 8* @ 00:36
- Video: *Lecture 8* @ 12:50
- Video: *Lecture 8* @ 16:20
- Video: *Lecture 8* @ 31:23
- Video: *Lecture 8* @ 14:57
- Video: *Lecture 8* @ 48:37
- Video: *Lecture 9* @ 06:08
- Video: *Lecture 9* @ 14:29
- Video: *Lecture 9* @ 21:18
- Video: *Lecture 9* @ 24:44
- Video: *Lecture 9* @ 50:51
- Video: *Video: 10* @ 05:29
- Video: *Video: 10* @ 20:34

- Video: *Video: 10* @ 12:58
- Video: *Video: 10* @ 28:50
- Video: *Video: 10* @ 43:20
- Video: *Video: 11* @ 04:05
- Video: *Video: 11* @ 08:14
- Video: *Video: 11* @ 13:42
- Video: *Video: 11* @ 42:37
- Video: *Video: 11* @ 46:02
- Video: *Video: 12* @ 01:23
- Video: *Video: 12* @ 93:34
- Video: *Lecture 13* @ 01:22
- Video: *Lecture 13* @ 03:26
- Video: *Lecture 14* @ 05:28
- Video: *Lecture 13* @ 17:15
- Video: *Lecture 13* @ 18:36
- Video: *Lecture 13* @ 22:02
- Video: *Lecture 13* @ 24:04
- Video: *Lecture 13* @ 06:11
- Video: *Lecture 13* @ 30:19
- Video: *Lecture 13* @ 36:28
- Video: *Lecture 13* @ 44:47
- Video: *Lecture 14* @ 03:21
- Video: *Lecture 14* @ 19:56
- Video: *Lecture 14* @ 26:48
- Video: *Lecture 14* @ 34:20
- Video: *Lecture 14* @ 45:18
- Video: *Lecture 14* @ 46:42
- Video: *Lecture 15* @ 09:04
- Video: *Lecture 17* @ 75:56
- Video: *Lecture 15* @ 62:09
- Video: *Lecture 15* @ 45:03
- Video: *Lecture 15* @ 49:12
- Video: *Lecture 16* @ 10:26
- Video: *Lecture 16* @ 27:44
- Video: *Lecture 16* @ 28:25
- Video: *Lecture 16* @ 33:17
- Video: *Lecture 13* @ 08:17
- Video: *Lecture 17* @ 20:44
- Video: *Lecture 17* @ 23:29
- Video: *Lecture 18* @ 02:04
- Video: *Lecture 18* @ 19:25

- Video: *Lecture 18 @ 25:38*
- Video: *Lecture 18 @ 52:39*
- Video: *Lecture 13 @ 24:45*
- Video: *Lecture 14 @ 09:33*
- Video: *Lecture 16 @ 11:06*
- Video: *Lecture 18 @ 08:17*
- Video: *Birds Eye View of the LLM Architecture @ 00:41*
- Video: *Birds Eye View of the LLM Architecture @ 04:46*
- Video: *Birds Eye View of the LLM Architecture @ 05:27*
- Video: *Birds Eye View of the LLM Architecture @ 12:18*
- Video: *Coding the entire LLM Transformer Block @ 17:17*
- Video: *Shortcut connections in the LLM Architecture @ 05:33*
- Video: *Layer Normalization in the LLM Architecture @ 09:42*
- Video: *Layer Normalization in the LLM Architecture @ 37:01*
- Video: *Layer Normalization in the LLM Architecture @ 30:04*
- Video: *Layer Normalization in the LLM Architecture @ 15:07*
- Video: *GELU Activation Function in the LLM Architecture @ 17:52*
- Video: *GELU Activation Function in the LLM Architecture @ 08:54*
- Video: *GELU Activation Function in the LLM Architecture @ 10:17*
- Video: *GELU Activation Function in the LLM Architecture @ 08:13*
- Video: *GELU Activation Function in the LLM Architecture @ 15:49*
- Video: *GELU Activation Function in the LLM Architecture @ 16:30*
- Video: *Shortcut connections in the LLM Architecture @ 10:20*
- Video: *Shortcut connections in the LLM Architecture @ 12:23*
- Video: *Shortcut connections in the LLM Architecture @ 15:13*
- Video: *Birds Eye View of the LLM Architecture @ 13:41*
- Video: *Coding the entire LLM Transformer Block @ 27:37*
- Video: *Coding the 124 million parameter GPT-2 model @ 33:43*
- Video: *Coding the 124 million parameter GPT-2 model @ 34:23*
- Video: *Coding GPT-2 to predict the next token @ 11:44*
- Video: *Coding GPT-2 to predict the next token @ 01:22*
- Video: *Coding GPT-2 to predict the next token @ 23:24*
- Video: *Saving and loading LLM model weights using PyTorch @ 02:40*
- Video: *Measuring the LLM loss function @ 00:42*
- Video: *Evaluating LLM performance on real dataset @ 09:36*
- Video: *Temperature Scaling in Large Language Models @ 13:04*
- Video: *Measuring the LLM loss function @ 35:40*
- Video: *Measuring the LLM loss function @ 42:27*
- Video: *Measuring the LLM loss function @ 51:32*
- Video: *Evaluating LLM performance on real dataset @ 03:25*
- Video: *Evaluating LLM performance on real dataset @ 05:28*

- Video: *Evaluating LLM performance on real dataset @ 11:40*
- Video: *Saving and loading LLM model weights using PyTorch @ 04:09*
- Video: *Evaluating LLM performance on real dataset @ 15:08*
- Video: *Evaluating LLM performance on real dataset @ 31:08*
- Video: *Evaluating LLM performance on real dataset @ 35:59*
- Video: *Evaluating LLM performance on real dataset @ 39:28*
- Video: *Temperature Scaling in Large Language Models @ 08:17*
- Video: *Temperature Scaling in Large Language Models @ 15:52*
- Video: *Top-k sampling in Large Language Models @ 06:15*
- Video: *Top-k sampling in Large Language Models @ 18:37*
- Video: *Saving and loading LLM model weights using PyTorch @ 07:32*
- Video: *Saving and loading LLM model weights using PyTorch @ 04:12*
- Video: *Saving and loading LLM model weights using PyTorch @ 37:13*
- Video: *Saving and loading LLM model weights using PyTorch @ 41:22*
- Video: *Saving and loading LLM model weights using PyTorch @ 44:53*
- Video: *Introduction to LLM Finetuning @ 00:42*
- Video: *Introduction to LLM Finetuning @ 02:06*
- Video: *Introduction to LLM Finetuning @ 01:45*
- Video: *Introduction to LLM Finetuning @ 04:12*
- Video: *Introduction to LLM Finetuning @ 08:19*
- Video: *Introduction to LLM Finetuning @ 09:44*
- Video: *Introduction to LLM Finetuning @ 16:38*
- Video: *Introduction to LLM Finetuning @ 19:24*
- Video: *Introduction to LLM Finetuning @ 20:04*
- Video: *Introduction to LLM Finetuning @ 20:46*
- Video: *Introduction to LLM Finetuning @ 22:09*
- Video: *Dataloaders in LLM Classification Finetuning @ 09:32*
- Video: *Dataloaders in LLM Classification Finetuning @ 06:47*
- Video: *Dataloaders in LLM Classification Finetuning @ 10:54*
- Video: *Dataloaders in LLM Classification Finetuning @ 12:16*
- Video: *Dataloaders in LLM Classification Finetuning @ 14:23*
- Video: *Dataloaders in LLM Classification Finetuning @ 15:45*
- Video: *Dataloaders in LLM Classification Finetuning @ 23:21*
- Video: *Dataloaders in LLM Classification Finetuning @ 27:30*
- Video: *Coding the model architecture @ 07:51*
- Video: *Coding the model architecture @ 06:50*
- Video: *Coding the model architecture @ 12:31*
- Video: *Coding the model architecture @ 18:43*
- Video: *Coding the model architecture @ 22:00*
- Video: *Coding the model architecture @ 23:15*
- Video: *Coding the model architecture @ 24:06*

- Video: *Coding the model architecture* @ 32:31
- Video: *Coding a fine-tuned LLM* @ 08:59
- Video: *Coding a fine-tuned LLM* @ 17:18
- Video: *Coding a fine-tuned LLM* @ 19:22
- Video: *Coding a fine-tuned LLM* @ 31:11
- Video: *Coding a fine-tuned LLM* @ 26:59
- Video: *Coding a fine-tuned LLM* @ 34:11
- Video: *Coding a fine-tuned LLM* @ 34:41
- Video: *Coding a fine-tuned LLM* @ 15:12, 33:59, 36:48
- Video: *Coding a fine-tuned LLM* @ 41:38
- Video: *Coding a fine-tuned LLM* @ 44:37
- Video: *Coding a fine-tuned LLM* @ 46:29
- Video: *Build LLMs from scratch 20 minutes summary* @ 02:46
- Video: *Introduction to LLM Instruction Fine-tuning* @ 02:06
- Video: *Introduction to LLM Instruction Fine-tuning* @ 02:47
- Video: *Introduction to LLM Instruction Fine-tuning* @ 03:29
- Video: *Introduction to LLM Instruction Fine-tuning* @ 04:11
- Video: *Introduction to LLM Instruction Fine-tuning* @ 05:34
- Video: *Introduction to LLM Instruction Fine-tuning* @ 09:44
- Video: *Data Batching in LLM instruction fine-tuning* @ 01:23
- Video: *Introduction to LLM Instruction Fine-tuning* @ 13:53
- Video: *Introduction to LLM Instruction Fine-tuning* @ 18:44
- Video: *Data Batching in LLM instruction fine-tuning* @ 12:25
- Video: *Data Batching in LLM instruction fine-tuning* @ 13:03
- Video: *Data Batching in LLM instruction fine-tuning* @ 15:50
- Video: *Data Batching in LLM instruction fine-tuning* @ 20:36
- Video: *Data Batching in LLM instruction fine-tuning* @ 44:01
- Video: *Instruction fine-tuning: Loading pre-trained LLM weights* @ 06:15
- Video: *Instruction fine-tuning: Loading pre-trained LLM weights* @ 07:37
- Video: *Instruction fine-tuning: Loading pre-trained LLM weights* @ 11:04
- Video: *LLM fine-tuning training loop* @ 11:11
- Video: *LLM fine-tuning training loop* @ 13:14
- Video: *LLM fine-tuning training loop* @ 20:47
- Video: *Data Batching in LLM instruction fine-tuning* @ 48:08
- Video: *Data Batching in LLM instruction fine-tuning* @ 49:32
- Video: *Evaluating fine-tuned LLM using Ollama* @ 13:51
- Video: *Evaluating fine-tuned LLM using Ollama* @ 15:54
- Video: *Evaluating fine-tuned LLM using Ollama* @ 16:36
- Video: *Evaluating fine-tuned LLM using Ollama* @ 36:47
- Video: *Evaluating fine-tuned LLM using Ollama* @ 24:58
- Video: *Evaluating fine-tuned LLM using Ollama* @ 28:26

- Video: *Evaluating fine-tuned LLM using Ollama* @ 40:57
- Video: *Evaluating fine-tuned LLM using Ollama* @ 44:47
- Video: *Evaluating fine-tuned LLM using Ollama* @ 47:08
- Video: *Evaluating fine-tuned LLM using Ollama* @ 49:10
- Video: *Build LLMs from scratch 20 minutes summary* @ 18:39